PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO
PÓS-GRADUAÇÃO EM CIÊNCIA DE DADOS E ANALYTICS
SPRINT ENGENHARIA DE DADOS

AVALIAÇÃO DOS ACIDENTES DE TRÂNSITO COM ÓBITOS EM RODOVIAS FEDERAIS NO BRASIL DE 2020 A 2025

Discente:
Max Brandão de Oliveira

FORTALEZA – CE
2025

PONTIFÍCIA UNIVERSIDADE CATÓLICA DO RIO DE JANEIRO
PÓS-GRADUAÇÃO EM CIÊNCIA DE DADOS E ANALYTICS
SPRINT ENGENHARIA DE DADOS

AVALIAÇÃO DOS ACIDENTES DE TRÂNSITO COM ÓBITOS EM RODOVIAS FEDERAIS NO BRASIL DE 2020 A 2025

Minimum Viable Product (MVP) da disciplina de Engenharia de Dados do curso Pós-Graduação em Ciência de Dados e Analytics da Pontifícia Universidade Católica do Rio de Janeiro (PUC – RIO) para obtenção do grau de especialista em Ciência de Dados e Analytics.

Discente: Max Brandão de Oliveira

FORTALEZA – CE
2025

Resumo

No Brasil, segundo os dados divulgados pela Confederação Nacional do Transporte, o número de acidentes nas rodovias federais brasileiras aumentou em 1,6% em 2021 em relação a 2020, totalizando 64.452 casos. Além disso, houve um aumento de 2,0% no número de mortes, passando de 5.287 em 2020 para 5.391 em 2021. Os dados de ocorrências de trânsito analisados neste estudo, disponibilizados em sítio público, foram estruturados a partir dos registros da Polícia Rodoviária Federal (PRF) ocorridos em toda malha rodoviária federal no período de 2020 a 2025. O objetivo deste trabalho foi identificar os fatores que estão relacionados às ocorrências de acidentes fatais nas rodovias federais do Brasil. Para alcançar este objetivo, ou seja, relacionar a variável resposta (óbito ou não) com as variáveis explicativas (condição de visibilidade, tipo de pista, estado, turno e outras), utilizou-se uma análise descritiva e inferencial, através do ajuste de uma regressão logística. Foi utilizada a plataforma databricks como repositório de dados na nuvem, bem como para a produção de *notebooks* contendo códigos nas linguagens sql e python para a obtenção e manuseio do banco de dados, bem como para a obtenção de medidas descritivas (contagem, proporção, média e desvio-padrão), teste qui-quadrado e estimação dos parâmetros da regressão logística. Foi adotada a arquitetura medalhão para o tratamento dos dados. Dentre os principais resultados, temos que quando a visibilidade é ruim, as chances de um óbito são 11,66% menores em relação à boa. Já a fase do dia, a chance de ocorrência de acidente fatal para o anoitecer em relação ao amanhecer reduz em 36,16%. Os dias com maiores incidências de óbitos são sábado e domingo. A pista simples apresenta mais de 76% de chance a mais de ocorrer óbito em relação à pista dupla. A ultrapassagem indevida aumenta em mais do dobro de chance de ocorrência de óbito, enquanto transitar na contramão aumenta em mais de 4 vezes a chance de óbito

Palavras-chave: Odds Ratio, acidentes de trânsito, arquitetura medalhão.

Sumário

Introdução 5

1. Metodologia 7

2. Arquitetura medalhão 13

2.1. Camada BRONZE 13

2.2. Camada SILVER 17

2.3. Camada GOLD 20

3. Resultados 29

4. Conclusão 33

5. Dificuldades e trabalhos futuros 34

Referências 34

Introdução

A segurança no trânsito é um aspecto de extrema relevância mundial. De acordo com a Organização Mundial da Saúde (OMS), estima-se que, a cada ano, cerca de 1,25 milhões de pessoas morrem no trânsito e até 50 milhões sobrevivem com ferimentos e sequelas, resultando em custos pessoais, sociais e econômicos.

Os acidentes de trânsito são uma das principais causas de morte entre jovens com idades entre 15 e 29 anos (OMS, 2018). Em 2020, para discutir estratégias para reduzir as mortes, a OMS organizou a terceira Conferência Ministerial Global sobre Segurança no Trânsito (OMS, 2020). No Brasil, de acordo com os dados divulgados pela Confederação Nacional do Transporte (CNT), o número de acidentes nas rodovias federais brasileiras aumentou em 1,6% em 2021 em relação a 2020, totalizando 64.452 casos. Além disso, houve um aumento de 2% no número de mortes, passando de 5.287 em 2020 para 5.391 em 2021, de acordo com os dados publicados pela Polícia Rodoviária Federal (PRF) (TRANSPORTE, CNT, 2021).

Esses acidentes oneram o sistema de saúde e a economia constantemente, bem como saturam a rede de emergência pública e privada. É preciso destacar que as sequelas permanentes decorrentes de acidentes de trânsito podem ter um impacto significativo na vida das pessoas, afetando sua capacidade de trabalho, lazer e convivência social. De acordo com informações divulgadas pelo Observatório Nacional de Segurança Viária (ONSV), uma organização dedicada a redução dos índices de acidentes de trânsito no Brasil, as internações decorrentes de acidentes rodoviários têm gerado um custo anual de R$ 52 bilhões para o sistema público de saúde. Esse valor equivale a 2,8% do Produto Interno Bruto (PIB) do país e leva em consideração despesas com tratamentos médicos, indenizações, perda de produtividade decorrente de lesões ou mortes e gastos associados aos veículos.

Esses valores evidenciam o impacto econômico causado pelos acidentes de trânsito na sociedade. Desta forma, é fundamental investir em políticas públicas que promovam a segurança no trânsito e o suporte adaptado às vítimas de acidentes. Identificar os fatores que podem contribuir para acidentes que resultam em óbito é fundamental para o desenvolvimento de estratégias eficazes destinadas a reduzir drasticamente o número de mortes.

Diante do exposto, os modelos de regressão possuem grande peso no que diz respeito a identificação destes fatores associados aos acidentes fatais, tendo um papel fundamental na determinação de fatores associados a acidentes fatais. Segundo **Agresti (2019)**, o modelo de regressão logística é o mais popular para dados binários, sendo que esse modelo é descrito como um Modelo Linear Generalizado (GLM) para um componente aleatório binomial com a função de ligação logit. Como exemplo disso, em seu Trabalho de Conclusão de Curso (TCC), **Rosemberg (2018)** aplicou a análise de regressão logística em seu estudo sobre a Incidência de Acidentes Fatais no Trânsito do Distrito Federal.

Este estudo tem como objetivo mapear os acidentes de trânsito com morte no Distrito Federal e ver onde os mesmos estavam mais propensos a ocorrer. No estudo de Morais et al. (2014) os autores utilizaram a regressão logística para estudar a letalidade dos acidentes de trânsito na cidade de Palmas-TO, o objetivo principal foi prever a incidência de acidentes de trânsito de acordo com sua gravidade, sem vítimas, com vítimas ou com vítimas fatais, utilizando o Modelo Logit Ordenado. Assim, os autores constataram que há maior probabilidade de acidentes ocorrem aos domingos, sendo que os acidentes envolvendo bicicletas e motocicletas apresentam o maior número de vítimas fatais.

Este estudo apresenta resultados que podem orientar os órgãos competentes a adotarem medidas preventivas e, consequentemente, reduzir a quantidade de acidentes fatais nas rodovias federais. Deseja-se que os resultados possam contribuir com a gestão da segurança rodoviária, dado que o Código de Trânsito Brasileiro (CTB) assegura que o trânsito em condições seguras é um direito de todos e dever dos órgãos e entidades componentes do Sistema Nacional de Trânsito (BRASIL, 1997), sendo uma importante atividade realizada pelos motoristas de vias e agentes de segurança pública.

Portanto, os resultados apresentados neste estudo podem ser valiosos para apoiar os esforços de gerenciamento de segurança rodoviária. Diante do exposto, o presente trabalho tem o objetivo de identificar fatores de risco relacionados às ocorrências de acidentes fatais nas rodovias brasileiras de 2020 a 2025 registrados pela PRF.

1. Metodologia

Os dados foram obtidos no sítio dados.gov.br/home (Figura 1) acessando o ícone referente ao "conjunto de dados abertos", que contém um total de 15.204 resultados de diversos temas e formatos. Na aba de pesquisa, ao digitar a palavra "PRF", foram exibidas 3 opções de dados abertos (Figura 2), dos quais optou-se pelos Sinistros de Trânsito fornecidos pela Polícia Rodoviária Federal (PRF), como observado na Figura 3.

Dentre os sinistros, os dados foram agrupados por Pessoa ou por Ocorrência, onde optou-se pela segunda opção (Figura 4), que fornecer arquivos no formato .csv disponibilizados por ano, de 2007 a 2025. No caso, foram feitos os *downloads* dos anos de 2020 a 2025, que contêm todas 30 variáveis colunas com as quantidades de linhas descritas na Tabela 1. Observe que no brasil, apenas nas rodovias federais, ocorreram em média 65.523 acidentes por ano. Entre os anos completos, de 2020 a 2024, o de menor valor foi o de 2020, talvez devido ao período da pandemia, que provavelmente reduziu o tráfego de veículos

Tabela 1: Quantidade de sinistros em rodovias federais de 2020 à 2025 segundo a PRF

Ano	2020	2021	2022	2023	2024	2025
Quantidade de sinistros	63.585	64.567	64.606	67.766	73.156	59.459

Figura 1: Sítio dados.gov.br/home para obtenção de dados abertos para implementação da pesquisa referente à MVP (*Minimum Viable Product*) da disciplina de Engenharia de Dados do curso de Pós-Graduação em Ciência de Dados e Analytics

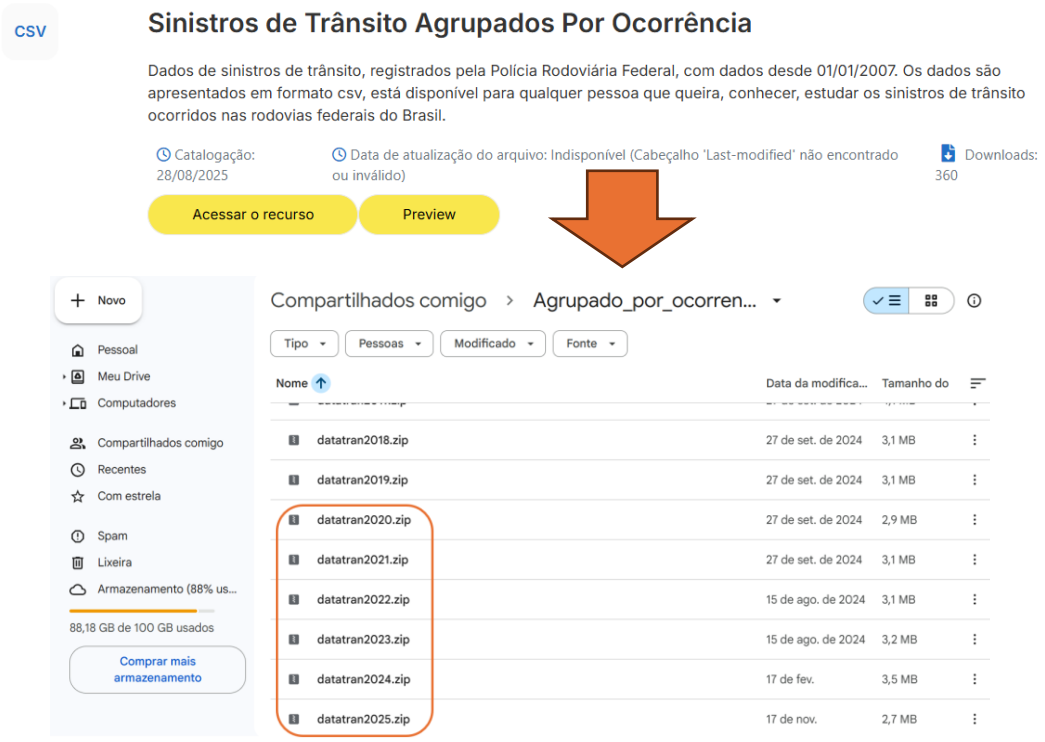
Figura 2: Sítio dados.gov.br/home para busca de “PRF” para obtenção de dados abertos para pesquisa referente à MVP (*Minimum Viable Product*) da disciplina de Engenharia de Dados do curso de Pós-Graduação em Ciência de Dados e Analytics

3 Conjuntos de dados abertos

Figura 3: Opção Sinistros de Trânsito fornecidos pela Polícia Rodoviária Federal (PRF) do sítio dados.gov.br/home para obtenção de dados abertos para MVP da disciplina de Engenharia de Dados da Pós-Graduação em Ciência de Dados e Analytics

O uso dos dados públicos permite que a própria sociedade tenha acesso e consiga observar fenômenos de interesse de acordo com o nicho da pesquisa. No caso em questão, a instituição pode utilizar os resultados para favorecer uma tomada de decisão mais consciente e objetiva, permitindo a elaboração de projetos que possam beneficiar políticas públicas de combate à sinistros em rodovias federais. Destaca-se que, devido o uso de uma máquina pessoal de baixo processamento, seria inviável usar os dados desde 2007.

Figura 4: Opção Sinistros de Trânsito por Ocorrência fornecidos pela PRF de 2007 a 2025 para obtenção de dados abertos para MVP da disciplina de Engenharia de Dados da Pós-Graduação em Ciência de Dados e Analytics



Após a obtenção dos dados brutos, foi realizado o cadastro na plataforma databricks através do sítio https://www.databricks.com/learn/free-edition (Figura 5), que é uma plataforma de análise de dados na nuvem que oferece um ambiente para trabalhar com *big data* e aprendizado de máquina. Além disso, é baseado em Apache Spark, permitindo processar grandes volumes de dados de forma rápida e eficiente. A Figura 6 a presenta os passos para o cadastro na plataforma e a Figura 7 mostra a página inicial com a conta "logada".

Figura 5: Página inicial do databricks para alocar os dados dos sinistros de trânsito por Ocorrência da PRF de 2007 a 2025 para obtenção de dados abertos para MVP da disciplina de Engenharia de Dados da Pós-Graduação em Ciência de Dados e Analytics

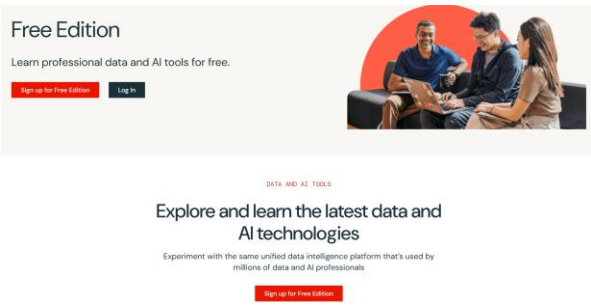


Figura 6: Passos para cadastro e login na plataforma databricks para alocação dos dados sobre os sinistros em rodovias federais de 2020 à 2025 para a MVP

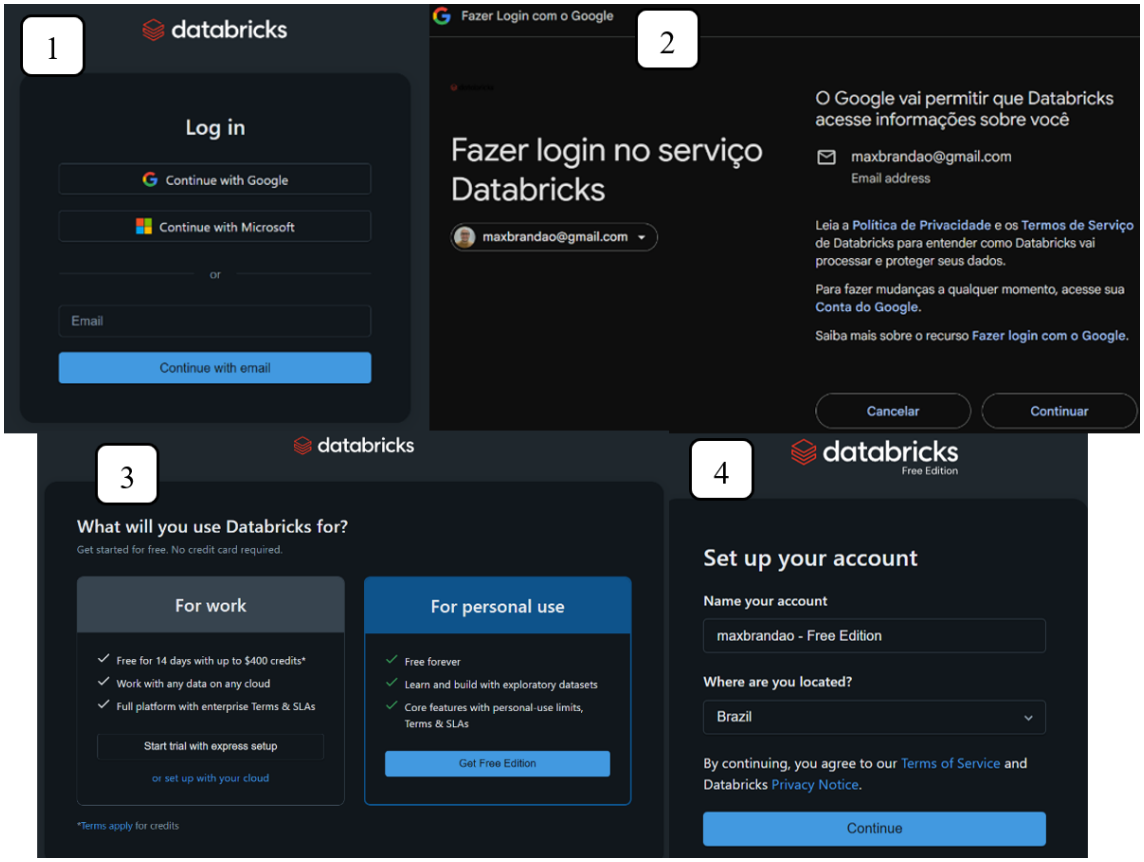
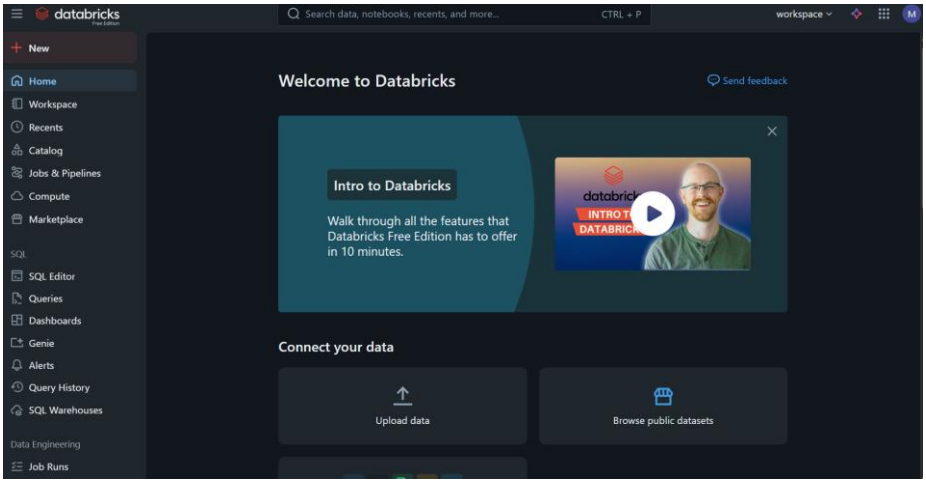


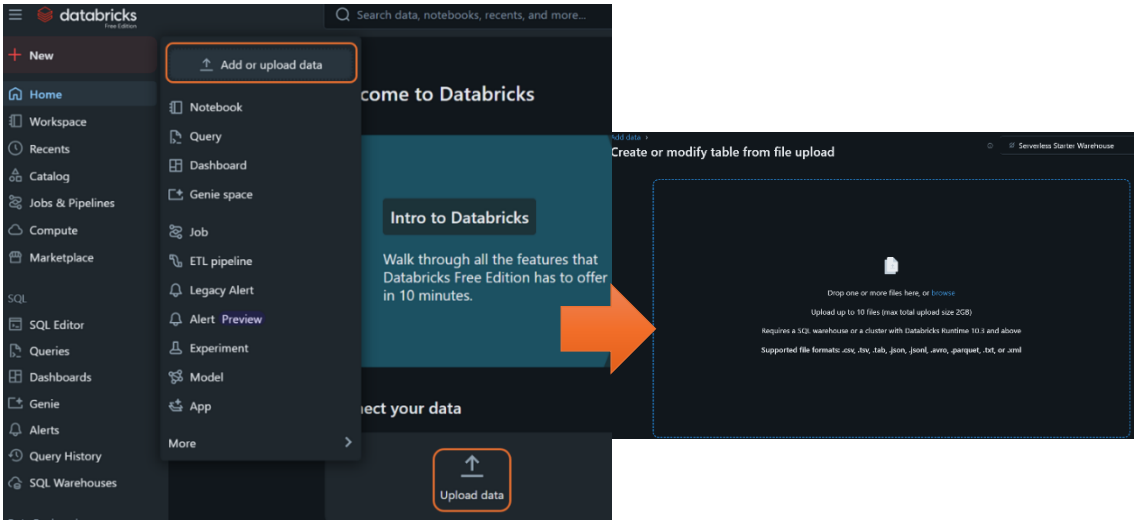
Figura 7: Acesso à plataforma databricks com a conta cadastrada para alocação dos dados sobre os sinistros em rodovias federais de 2020 à 2025 para a MVP



Já com acesso à plataforma, podemos inserir os 6 bancos de dados no formato .csv denominados de “datatran2020.csv”, “datatran2021.csv”, “datatran2022.csv”,

“datatran2023.csv”, “datatran2024.csv” e “datatran2025.csv”, seguimos a sequência apresentada na Figura 8 para inserir os 6 arquivos na nuvem.

Figura 8: Upload no databricks dos 6 arquivos referentes aos sinistros de trânsito de 2020 a 2025 disponibilizados pela Polícia Rodoviária Federal (PRF)



A Figura 9 exibe os arquivos originais baixados no sítio dados.gov.br já disponíveis no databricks para manuseio e organizados na pasta intitulada “MVP_1”. Na mesma pasta foi criado o notebook para leitura e tratamento dos arquivos, como observado na figura 10. Perceba que os arquivos originais foram baixados e imediatamente inseridos na plataforma, sem sequer serem abertos.

Na leitura inicial, percebeu-se que os dados usavam “,” como separador decimal e eram separados por “;”. Para tanto, foi necessário indicar na função spark.read.csv com os atributos sep=";", que indica “;” como separador entre as colunas; inferSchema=True, que sugere automaticamente o formato das variáveis; e encoding='ISO-8859-1', por conta dos caracteres especiais de acentuação, que o padrão ‘utf-8’ não reconhece, fazendo com que as palavras com acentuação aparecessem com erro.

Os 6 dados arquivos referentes a cada ano foram agrupado em um único arquivo chamado datatran_all, bem como foi acrescentada uma coluna referente ao ano em que cada acidente ocorreu, de modo que passou a ter 31 variáveis (colunas) e 393.139 ocorrências (linhas). Os dados contidos no objeto datatran_all são associados à fase bronze na arquitetura medalhão (*Medallion Architecture*).

Figura 9: Dados brutos no formato .csv disponíveis no databricks referentes aos sinistros de trânsito de 2020 a 2025 disponibilizados pela PRF

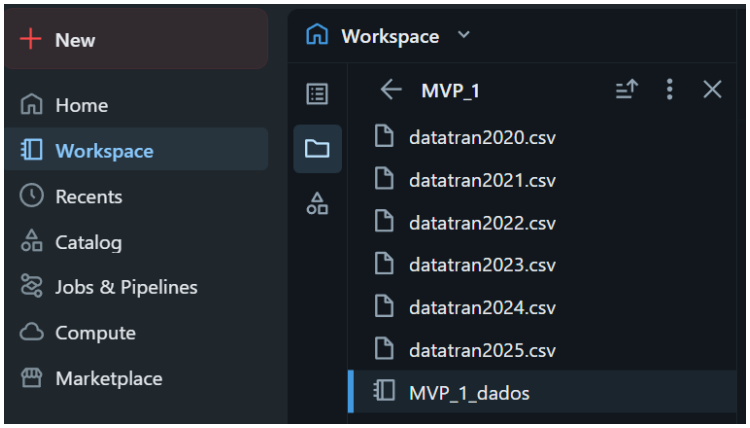


Figura 10: Comandos para leitura dos 6 bancos de dados carregados no databricks referentes aos sinistros de trânsito de 2020 a 2025 disponibilizados pela PRF

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import lit, regexp_replace, col

# Criar SparkSession
spark = SparkSession.builder \
    .appName("Nome da sua Aplicação") \
    .getOrCreate()

# Caminho dos arquivos
file_paths = [
    ('/Workspace/Users/maxbrandao@gmail.com/MVP_1/datatran2020.csv', 2020),
    ('/Workspace/Users/maxbrandao@gmail.com/MVP_1/datatran2021.csv', 2021),
    ('/Workspace/Users/maxbrandao@gmail.com/MVP_1/datatran2022.csv', 2022),
    ('/Workspace/Users/maxbrandao@gmail.com/MVP_1/datatran2023.csv', 2023),
    ('/Workspace/Users/maxbrandao@gmail.com/MVP_1/datatran2024.csv', 2024),
    ('/Workspace/Users/maxbrandao@gmail.com/MVP_1/datatran2025.csv', 2025)]

# Lendo e processando os arquivos
dataframes = []
for file_path, year in file_paths:
    datatran = spark.read.csv(file_path, sep=";", inferSchema=True, header=True,encoding='ISO-8859-1')
    # Transformando várias colunas
    datatran = datatran.withColumn("km", regexp_replace(col("km"), ",", ".").cast("double")) \
                       .withColumn("latitude", regexp_replace(col("latitude"), ",", ".").cast("double")) \
                       .withColumn("longitude", regexp_replace(col("longitude"), ",", ".").cast("double")) \
                       .withColumn("ano", lit(year))

    dataframes.append(datatran)
# Unindo todos os DataFrames
datatran_all = dataframes[0]
for datatran in dataframes[1:]:
    datatran_all = datatran_all.unionByName(datatran)

# Exibindo os 5 primeiros registros do DataFrame combinado
datatran_all.show(5)
```

2. Arquitetura medalhão

2.1. Camada BRONZE

Para conferência, temos a figura 11 contendo as 5 primeiras linhas dos dados do daa frame datatran_all, assim como a figura 12 apresenta os comandos em sql para a criação do CATALOG intitulado mvp1, do SCHEMA bronze e do VOLUME datatran. Já as figuras 13 e 14 exibem como o data frame datatran_all foi salvo com o respectivo dicionário de dados. O quando 1 exibe o dicionário com dos dados da arquitetura broze.

Figura 11: Conferência do número de linhas e de colunas e exibição dos resultados

```
# Obtendo o número de linhas e de colunas
num_linhas = datatran_all.count()
num_colunas = len(datatran_all.columns)
# Exibindo as dimensões
print(f"Número de linhas: {num_linhas}, Número de colunas: {num_colunas}")
# Exibindo os dados
display(datatran_all.limit(5))
> See performance (3)
> datatran: pyspark.sql.connect.dataframe.DataFrame = [id: integer, data_inversa: date ... 29 more fields]
> datatran_all: pyspark.sql.connect.dataframe.DataFrame = [id: double, data_inversa: date ... 29 more fields]
Número de linhas: 393139, Número de colunas: 31
```

Table

	1.2 id	data_inversa	dia_semana	horario	uf	br	1.2
1	260068	2020-01-01	quarta-feira	2025-12-15T05:40:00.000+00:00	PA	316	
2	260073	2020-01-01	quarta-feira	2025-12-15T06:00:00.000+00:00	MG	262	
3	260087	2020-01-01	quarta-feira	2025-12-15T06:00:00.000+00:00	BA	116	
4	260116	2020-01-01	quarta-feira	2025-12-15T10:08:00.000+00:00	SP	116	
5	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	[illegible]	

Figura 12: Criação do CATALOG mvp1, do SCHEMA bronze e do VOLUME datatran

```
✓ 10:31 PM (<1s)                    2: 2. CATALOG mvp1
%sql
CREATE CATALOG IF NOT EXISTS mvp1
> See performance (1)
OK

✓ 4 minutes ago (<1s)               3: 3. SCHEMA bonze
%sql
CREATE SCHEMA IF NOT EXISTS mvp1.bronze
> See performance (1)
OK

✓ 2 minutes ago (1s)                4: 4. VOLUME datatran
%sql
CREATE VOLUME IF NOT EXISTS mvp1.bronze.datatran
> See performance (1)
OK
```

Figura 13: Geração da tabela do esquema medalhão bronze a partir do data frame criado em python

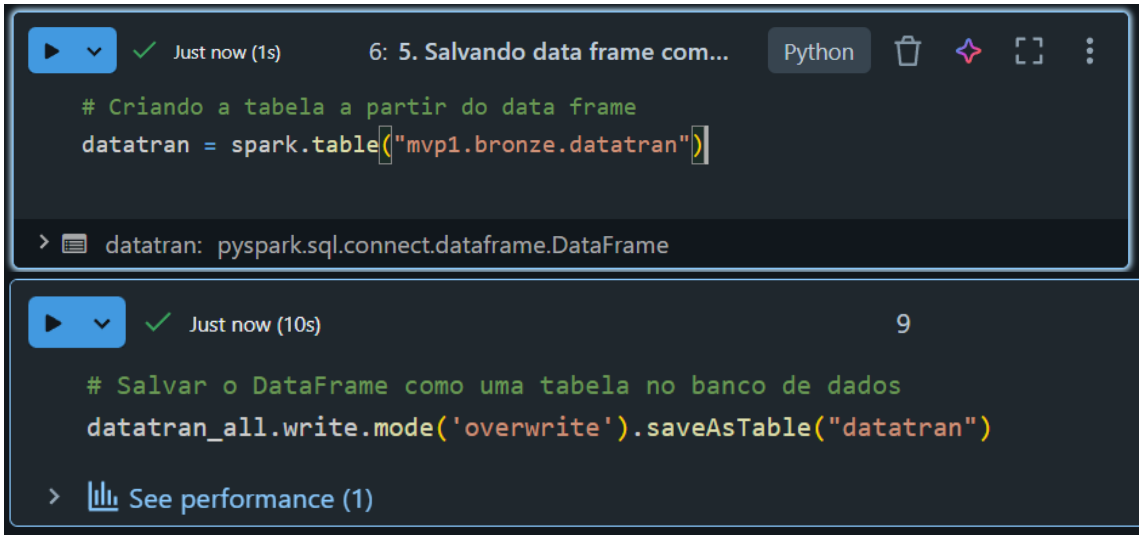
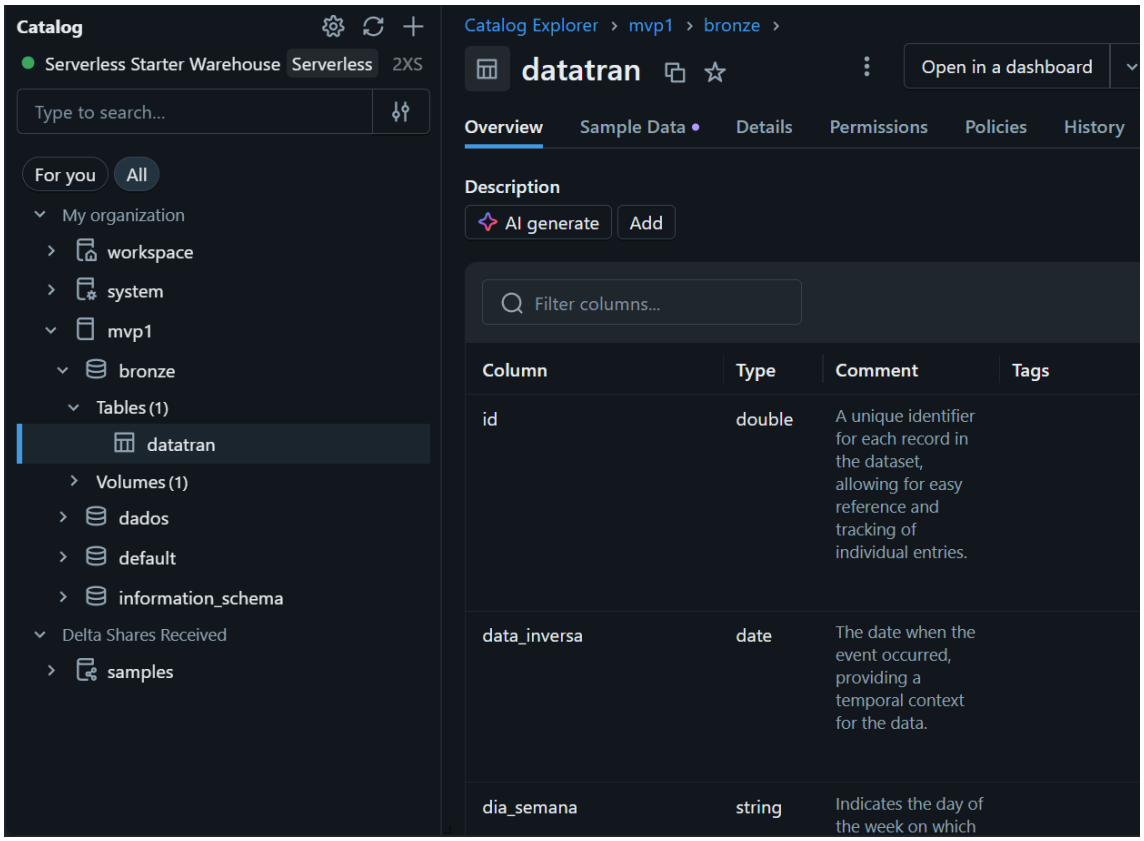


Figura 14: Exibição da tabela (table) gerada com base na figura 13 com o respectivo dicionário de dados com os comentários gerados pela IA disponível na plataforma



Quadro 1: Dicionário da tabela datatran_all da camada bronze da arquitetura medalhão

Variável	Tipo	Comentário
id	double	Um identificador único para cada registro no conjunto de dados, permitindo fácil referência e rastreamento de entradas individuais.
data_inversa	date	A data em que o evento ocorreu, fornecendo um contexto temporal para os dados.
dia_semana	string	O horário exato em que o evento ocorreu, permitindo uma análise temporal detalhada.
horario	timestamp	O horário exato em que o evento ocorreu, permitindo uma análise temporal detalhada.
uf	string	Representa o estado onde o evento ocorreu, o que é essencial para a análise regional.
br	int	O número da rodovia federal associado ao evento, útil para identificar rotas específicas.
km	double	O marcador quilométrico na rodovia onde o incidente ocorreu fornece informações precisas sobre a localização.
municipio	string	O município onde o evento ocorreu, auxiliando na análise e na elaboração de relatórios locais.
causa_acidente	string	Descreve a causa do acidente, o que é fundamental para compreender os fatores que contribuíram para ele.
tipo_acidente	string	Classifica o tipo de acidente, ajudando a identificar tendências e cenários comuns.
classificacao_acidente	string	Classifica a gravidade do acidente, o que é importante para avaliar o impacto e as necessidades de resposta.
fase_dia	string	Indica a fase do dia (por exemplo, manhã, tarde, noite) em que o evento ocorreu, útil para análises baseadas no tempo.

sentido_via	string	Descreve a direção do fluxo de tráfego na estrada no momento do evento, o que pode influenciar a dinâmica do acidente.
condicao_metereologica	string	Descreve detalhadamente as condições meteorológicas no momento do evento, que podem ser um fator significativo na ocorrência de acidentes.
tipo_pista	string	Especifica o tipo de superfície da estrada (por exemplo, asfalto, cascalho) onde o evento ocorreu, relevante para avaliações de segurança.
tracado_via	string	Descreve o traçado da estrada (por exemplo, reta, curva), o que pode influenciar as taxas de acidentes.
uso_solo	string	Indica o tipo de uso do solo na área ao redor do evento, o que pode fornecer contexto para os padrões de tráfego.
pessoas	int	O número total de pessoas envolvidas no evento é importante para entender a dimensão do incidente.
mortos	int	O número de fatalidades resultantes do evento é uma métrica crucial para avaliar a gravidade dos acidentes.
feridos_leves	int	Contabiliza o número de indivíduos com ferimentos leves, o que é útil para compreender o impacto nas vítimas.

Ainda na camada bronze, o arquivo “datatran_fatal” foi salvo considerando um número menor de variáveis, no caso, apenas 11 conforme a seleção apresentada na figura 15 (id, dia_semana, uf, causa_acidente, tipo_acidente, classificacao_acidente, fase_dia, condicao_metereologica, tipo_pista, tracado_via, mortos) e apenas ocorrência que tiveram pelo menos um óbito. A tabela “datatran_fatal” contém 28.217 linhas, indicando

que entre 2020 e 2025, tiveram 28.217 ocorrências com vítimas fatais em rodovias federais.

Figura 15: Criação de uma nova tabela apenas para acidentes com pelo menos uma vítima fatal (id, dia_semana, uf, causa_acidente, tipo_acidente, classificacao_acidente, fase_dia, condicao_metereologica, tipo_pista, tracado_via, mortos)

```
Just now (4s)        10: 10. Selecionando as variáveis (co...        SQL
%sql
CREATE TABLE mvp1.bronze.datatran_fatal AS
SELECT id, dia_semana, uf, causa_acidente, tipo_acidente, classificacao_acidente,
fase_dia, condicao_metereologica, tipo_pista, tracado_via, mortos
FROM datatran
WHERE mortos >= 1;
See performance (1)                                                    Optimize
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long, num_inserted_rows:
long]
```

Figura 16: Seleção de dados decorrentes de acidentes com óbitos entre 2020 e 2025

```
1 minute ago (35s)        11: 11. Contagem de ocorrÊncias co
%sql
SELECT COUNT(*) FROM datatran_fatal;
See performance (1)
_sqldf: pyspark.sql.connect.dataframe.DataFrame = [COUNT(*): long]
Table  +
```

	COUNT(*)
1	28217

2.2. Camada SILVER

A figura 17 apresenta os comandos em sql para a criação do SCHEMA silver, que tem o objetivo de fazer um recorte dos dados que serão de fato relevantes para a pesquisa no sentido de atender os objetivos da pesquisa. Já a figura 18 contém o código para seleção das 12 variáveis mais importantes para a pesquisa ("id", "uf", "visibilidade", "fase_dia","dia_semana", "causa_acidente", "uso_solo", "tipo_pista", "tipo_acidente", "obito", "ano", "classificacao_acidente"), de modo que o dicionário do quadro tem o mesmo resultado para tais características. Foram acrescentas a variável “obito”, que é binária e indica se teve ou não morte na ocorrência, assim como “visibilidade” indica se

a visibilidade no momento da ocorrência é boa ou não e baseia-se a “condição_meteorologica”.

Figura 17: Criação do SCHEMA silver no CATALOG mvp1 e da tabela datatran_clean

```
3 minutes ago (1s)                    12: 12. Criando SCHEMA silver
%sql
CREATE SCHEMA IF NOT EXISTS mvp1.silver
See performance (1)
_sqldf: pyspark.sql.connect.dataframe.DataFrame
OK
This result is stored as _sqldf and can be used in other Python and SQL cells.

Just now (1s)                         13: 13. Criando volume silver
%sql
CREATE VOLUME IF NOT EXISTS mvp1.silver.datatran_clean
See performance (1)
_sqldf: pyspark.sql.connect.dataframe.DataFrame
OK
This result is stored as _sqldf and can be used in other Python and SQL cells.
```

Figura 18: Comandos python spark para a seleção do banco de dados final limpo e codificado com 12 variáveis mais relevantes para o número de ocorrências de acidentes de trânsito nas rodovias federais de 2020 a 2025

```
Just now (2s)                         14: 14. Limpando e organizando a tabela final na ar...   Python
from pyspark.sql import SparkSession
from pyspark.sql.functions import when, col
# Initialize Spark session
spark = SparkSession.builder \
    .appName("Data Transformation") \
    .getOrCreate()
# Assuming datatran_all is already a Spark DataFrame
# Mutate 'visibilidade' column based on 'condicao_metereologica'
datatran_all = spark.sql("SELECT * FROM mvp1.bronze.datatran")
datatran_all = datatran_all.withColumn( "visibilidade",
    when(col("condicao_metereologica") == "Céu Claro", "bom")
    .when(col("condicao_metereologica") == "Nublado", "bom")
    .when(col("condicao_metereologica") == "Sol", "bom")
    .when(col("condicao_metereologica") == "Vento", "bom")
    .when(col("condicao_metereologica") == "Chuva", "ruim")
    .when(col("condicao_metereologica") == "Garoa/Chuvisco", "ruim")
    .when(col("condicao_metereologica") == "Granizo", "ruim")
    .when(col("condicao_metereologica") == "Neve", "ruim")
    .when(col("condicao_metereologica") == "Nevoeiro/Neblina", "ruim")
    .when(col("condicao_metereologica") == "Ignorado", None) )
# Transformando 'mortos' em 'obito' e recategorizando o string
datatran_all = datatran_all.withColumn( "obito", when(col("mortos") > 0, "sim").otherwise("não"))
# Selecionando as variáveis e renomeando o data.frame
datatran_silver = datatran_all.select("id","uf","visibilidade","fase_dia","dia_semana",
    "causa_acidente","uso_solo","tipo_pista","tipo_acidente","obito","ano","classificacao_acidente")

datatran_silver.display(5)
See performance (1)
datatran_all: pyspark.sql.connect.dataframe.DataFrame = [id: double, data_inversa: date ... 31 more fields]
datatran_silver: pyspark.sql.connect.dataframe.DataFrame = [id: double, uf: string ... 10 more fields]
```

Enquanto a figura 19 exibe o uso do catalog mvp1, do schema silver e da alocação do data frame datatran_silver à tabela datatran_clean, a figura 20 apresenta a criação do de uma função (exibir_percentuais_contingencia) para exibir e gerar tabelas de contingência percentuais com o teste qui-quadrado para avaliar a relação estatística entre as variáveis qualitativas.

Figura 19: Criação da tabela datatran_clean no SCHEMA silver do CATALOG mvp1

```
05:01 PM (1s)                    15: 15. CATALOG mpv1
%sql
USE CATALOG mvp1
See performance (1)
OK
This result is stored as _sqldf and can be used in other Python and SQL cells.

05:01 PM (1s)                    16: 16. SCHEMA silver                    SQL
%sql
USE SCHEMA silver
See performance (1)
_sqldf: pyspark.sql.connect.dataframe.DataFrame
OK
This result is stored as _sqldf and can be used in other Python and SQL cells.

1 minute ago (<1s)               17: 17. Criando tabela a do data frame
# Criando a tabela a partir do data frame
datatran_clean = spark.table("mvp1.silver.datatran_clean")
datatran_clean: pyspark.sql.connect.dataframe.DataFrame

Just now (5s)                    18: 18. Salvando a tabela final no schema silver    Python
# Salvar o DataFrame como uma tabela no banco de dados
datatran_silver.write.mode('overwrite').saveAsTable("mvp1.silver.datatran_clean")
```

A partir da tabela datatran_clean, que representa a camada silver com dados limpos, podemos utilizá-la como referência para obter os resultados definitivos para o objetivo da pesquisa. Em outras palavras, será possível gerar resultados que impliquem diretamente na obtenção de tabelas que respondam os objetivos da pesquisa, que se refere à camada ouro. A camada outro é capaz de usar tabelas para responder os questionamentos indagados que refletem o objetivo da pesquisa.

Figura 20: Criação de uma função em python spark para gerar tabelas de contingência com base nas variáveis de interesse com o teste qui-quidrado para a avaliar relação estatística entre as características

```python
3 minutes ago (15s)                    19: 19. Tabelas de contingência                    Python

from pyspark.sql.functions import sum as spark_sum, round, col
import pandas as pd
from scipy.stats import chi2_contingency

def exibir_percentuais_contingencia(df, index_col, pivot_col):
    # Exclui observações faltantes em index_col ou pivot_col
    df = df.filter(col(index_col).isNotNull() & col(pivot_col).isNotNull())
    # Cria tabela de contingência
    cont_table = df.groupBy(index_col, pivot_col).count() \
        .groupBy(index_col).pivot(pivot_col).sum("count")
    # Calcula total por linha
    total_col = "total"
    cont_table = cont_table.withColumn(total_col, sum([col(c) for c in cont_table.columns if c != index_col]))
    # Calcula percentuais por linha, excluindo colunas nulas
    percent_exprs = [
        (round(col(c) / col(total_col) * 100, 2).alias(c))
        for c in cont_table.columns if c not in [index_col, total_col] and c.lower() != "null"
    ]
    percent_table = cont_table.select(index_col, *percent_exprs).orderBy(index_col)
    display(percent_table)
    # Calcula valor p do teste qui quadrado
    cont_pd = cont_table.drop(total_col).toPandas().set_index(index_col)
    cont_pd = cont_pd.fillna(0)
    chi2, p, dof, expected = chi2_contingency(cont_pd.values)
    print(f"Valor p do teste qui quadrado: {p:.4g}")

# Tabelas de contingência
exibir_percentuais_contingencia(datatran_silver, "obito", "visibilidade")
exibir_percentuais_contingencia(datatran_silver, "uf", "obito")
> See performance (4)
```

2.3. Camada GOLD

A figura 21 mostra um resultado importante que já nos fornece ideia da relação entre a ocorrência de óbito no acidente com a visibilidade, que pelo teste qui-quadrado, ao nível de significância de 5%, indica que há uma relação de dependência entre tais características. Invertendo o pivô de comparação, a relação é perceptível e permanece. Esse resultado, já é um resultado a ser atribuído à camada gold como um resultado que atende diretamente o objetivo.

Já na figura 22, onde é observado o óbito por estado, percebemos que as unidades federativas com mais óbitos nas ocorrências foram Amazonas e Pará, com mais de 17% dos acidentes com vítimas fatais. Desta forma, é necessário avaliar o porquê de tal fenômeno para atacá-lo, seja com ações de monitoramento ou com correções estruturais nas rodovias. Já São Paulo e Distrito Federal foram os de menores percentuais.

Figura 21: Tabela de contingência entre óbito e visibilidade no momento da ocorrência do acidente de 2020 a 2025 em rodovias federais

Table

	obito	1.2 bom	1.2 ruim
1	não	85.25	14.75
2	sim	86.48	13.52

2 rows | 14.55s runtime

Valor p do teste qui quadrado: 2.709e-08

Table

	visibilidade	1.2 não	1.2 sim
1	bom	92.76	7.24
2	ruim	93.41	6.59

Figura 22: Tabela de contingência entre óbito e estado no momento da ocorrência do acidente de 2020 a 2025 em rodovias federais

Table

	uf	1.2 não	1.2 sim
1	AC	93.45	6.55
2	AL	88.71	11.29
3	AM	88.71	11.29
4	AP	93.65	6.35
5	BA	88.12	11.88
6	CE	89.72	10.28
7	DF	96.18	3.82
8	ES	94.39	5.61
9	GO	92.42	7.58
10	MA	82.09	17.91
11	MG	93.06	6.94
12	MS	92.03	7.97
13	MT	91.39	8.61
14	PA	82.86	17.14
15	PB	93.03	6.97

27 rows | 14.55s runtime

Valor p do teste qui quadrado: 0

Table

	uf	1.2 não	1.2 sim
13	MT	91.39	8.61
14	PA	82.86	17.14
15	PB	93.03	6.97
16	PE	90.21	9.79
17	PI	89.33	10.67
18	PR	93.29	6.71
19	RJ	94.78	5.22
20	RN	93.33	6.67
21	RO	94.39	5.61
22	RR	87.72	12.28
23	RS	94.49	5.51
24	SC	95.73	4.27
25	SE	92.37	7.63
26	SP	95.37	4.63
27	TO	87.67	12.33

27 rows | 8.11s runtime

Valor p do teste qui quadrado: 0

Figura 23: Código para obtenção das tabelas de contingência entre óbito com os demais fatores no momento da ocorrência do acidente de 2020 a 2025 em rodovias federais

```
10:00 PM (16s)                                  20: 20. Tabelas de contingência

# Tabelas de contingência
exibir_percentuais_contingencia(datatran_silver, "ano","obito")
exibir_percentuais_contingencia(datatran_silver, "fase_dia", "obito")
exibir_percentuais_contingencia(datatran_silver, "dia_semana", "obito")
exibir_percentuais_contingencia(datatran_silver, "causa_acidente", "obito")
exibir_percentuais_contingencia(datatran_silver, "uso_solo", "obito")
exibir_percentuais_contingencia(datatran_silver, "tipo_pista", "obito")
exibir_percentuais_contingencia(datatran_silver, "tipo_acidente", "obito")
exibir_percentuais_contingencia(datatran_silver, "classificacao_acidente", "obito")
> See performance (16)
```

Figura 24: Tabela de contingência entre óbito e fase do dia no momento da ocorrência do acidente de 2020 a 2025 em rodovias federais

Table

	fase_dia	1.2 não	1.2 sim
1	Amanhecer	89.39	10.61
2	Anoitecer	93.29	6.71
3	Plena Noite	89.94	10.06
4	Pleno dia	94.93	5.07

4 rows | 18.19s runtime

```
Valor p do teste qui quadrado: 0
```

Figura 25: Tabela de contingência entre óbito e dia da semana no momento da ocorrência do acidente de 2020 a 2025 em rodovias federais

Table

	dia_semana	1.2 não	1.2 sim
1	domingo	91.22	8.78
2	quarta-feira	93.58	6.42
3	quinta-feira	93.34	6.66
4	segunda-feira	93.43	6.57
5	sexta-feira	93.18	6.82
6	sábado	92	8
7	terça-feira	93.6	6.4

7 rows | 18.19s runtime

```
Valor p do teste qui quadrado: 9.675e-97
```

Figura 26: Tabela de contingência entre óbito e tipo de pista no momento da ocorrência do acidente de 2020 a 2025 em rodovias federais

Table

	tipo_pista	1.2 não	1.2 sim
1	Dupla	95.22	4.78
2	Múltipla	95.94	4.06
3	Simples	90.21	9.79

3 rows | 18.19s runtime

```
Valor p do teste qui quadrado: 0
```

Figura 27: Tabela de contingência entre óbito e tipo de acidente no momento da ocorrência do acidente de 2020 a 2025 em rodovias federais

	tipo_acidente	1.2 não	1.2 sim
1	Atropelamento de Animal	93.68	6.32
2	Atropelamento de Pedestre	70.88	29.12
3	Capotamento	96.24	3.76
4	Colisão com objeto	94.9	5.1
5	Colisão com objeto em movimento	97.25	2.75
6	Colisão com objeto estático	94.54	5.46
7	Colisão frontal	70.91	29.09
8	Colisão lateral	96.7	3.3
9	Colisão lateral mesmo sentido	97.32	2.68
10	Colisão lateral sentido oposto	91.46	8.54
11	Colisão transversal	95.39	4.61
12	Colisão traseira	95.8	4.2
13	Danos eventuais	97.84	2.16
14	Derramamento de carga	97.54	2.46
15	Engavetamento	97.93	2.07

21 rows | 18.19s runtime

```
Valor p do teste qui quadrado: 0
```

Figura 28: Tabela de contingência entre óbito e ano no momento da ocorrência do acidente de 2020 a 2025 em rodovias federais

Table

	ano	não	sim
1	2020	92.88	7.12
2	2021	92.77	7.23
3	2022	92.78	7.22
4	2023	92.83	7.17
5	2024	92.86	7.14
6	2025	92.8	7.2

6 rows | 16.09s runtime

Valor p do teste qui quadrado: 0.9642

As figuras 24, 25, 26, 27 e 28 relacionam a ocorrência de óbito fase do dia, dia da semana e tipo de pista, tipo de acidente e ano, respectivamente. Nos 4 primeiros cenários houve relação estatística com os fatores analisados com a ocorrência de óbito. Destaca-se que na pista simples, ocorreram acidentes fatais em quase 10% das ocorrências, enquanto sábado e domingo foram os dias com maiores percentuais de acidentes fatais. No caso, como esperado, não foi detectada relação estatística com os anos (figura 28).

Figura 29: Criação do SCHEMA gold e do VOLUME obito_uf no CATALOG mvp1

1 minute ago (1s) 21: 21. Criando o SCHEMA gold

```
%sql
CREATE SCHEMA IF NOT EXISTS mvp1.gold
```

See performance (1)

_sqldf: pyspark.sql.connect.dataframe.DataFrame

OK

This result is stored as _sqldf and can be used in other Python and SQL cells.

Just now (1s) 22: 22. Criando o VOLUME obbito_uf

```
%sql
CREATE VOLUME IF NOT EXISTS mvp1.gold.obito_uf
```

See performance (1)

_sqldf: pyspark.sql.connect.dataframe.DataFrame

OK

This result is stored as _sqldf and can be used in other Python and SQL cells.

Já na camada gold, podemos salvar a tabela apresentada figura 22, que retrata a contagem de acidentes com óbitos por estado, usando a linguagem pura na sintaxe “sql”, como podemos observar na figura 30. O mesmo pode ser feito por ano, visibilidade e dia da semana, como mostra a figura 31 e é confirmado pela figura 32 ao exibir as tabelas do schema gold.

Destaca-se que, após geradas as tabelas e alocadas no schema gold, é possível fazer gráficos e análises mais apuradas dependendo do interesse e do formato da apresentação dos dados, como mostram as figuras 33 e 34.

Figura 30: Criação da tabela de contagem entre acidentes com óbitos por estado nas rodovias federais de 2020 a 2025

Just now (6s) 23

```
%sql
CREATE OR REPLACE TABLE mvp1.gold.obito_uf AS
SELECT uf,
       SUM(CASE WHEN obito = 'sim' THEN 1 ELSE 0 END) AS obito_sim,
       SUM(CASE WHEN obito = 'não' THEN 1 ELSE 0 END) AS obito_nao
FROM mvp1.silver.datatran_clean
GROUP BY uf
ORDER BY uf;

SELECT * FROM mvp1.gold.obito_uf;
```

See performance (2)

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [uf: string, obito_sim: long ... 1 more field]

Table

	uf	obito_sim	obito_nao
1	AC	99	1413
2	AL	414	3252
3	AM	83	652
4	AP	60	885

Figura 31: Criação das tabelas de contagem entre acidentes com óbitos por visibilidade, por ano e por dia da semana nas rodovias federais de 2020 a 2025

1 minute ago (9s) 24: Tabelas na camada gold para ano, visibilidade e...

```
%sql
CREATE OR REPLACE TABLE mvp1.gold.obito_visibilidade AS
SELECT visibilidade,
       SUM(CASE WHEN obito = 'sim' THEN 1 ELSE 0 END) AS obito_sim,
       SUM(CASE WHEN obito = 'não' THEN 1 ELSE 0 END) AS obito_nao
FROM mvp1.silver.datatran_clean
GROUP BY visibilidade
ORDER BY visibilidade;

CREATE OR REPLACE TABLE mvp1.gold.obito_ano AS
SELECT ano,
       SUM(CASE WHEN obito = 'sim' THEN 1 ELSE 0 END) AS obito_sim,
       SUM(CASE WHEN obito = 'não' THEN 1 ELSE 0 END) AS obito_nao
FROM mvp1.silver.datatran_clean
GROUP BY ano
ORDER BY ano;

SELECT * FROM mvp1.gold.obito_ano;

CREATE OR REPLACE TABLE mvp1.gold.obito_dia AS
SELECT dia_semana,
       SUM(CASE WHEN obito = 'sim' THEN 1 ELSE 0 END) AS obito_sim,
       SUM(CASE WHEN obito = 'não' THEN 1 ELSE 0 END) AS obito_nao
FROM mvp1.silver.datatran_clean
GROUP BY dia_semana
ORDER BY dia_semana;
```

Figura 32: Confirmação da criação das tabelas de contagem entre acidentes com óbitos por visibilidade, por ano e por dia da semana nas rodovias federais de 2020 a 2025 no CATALOG mvp1 e no SCHEMA gold

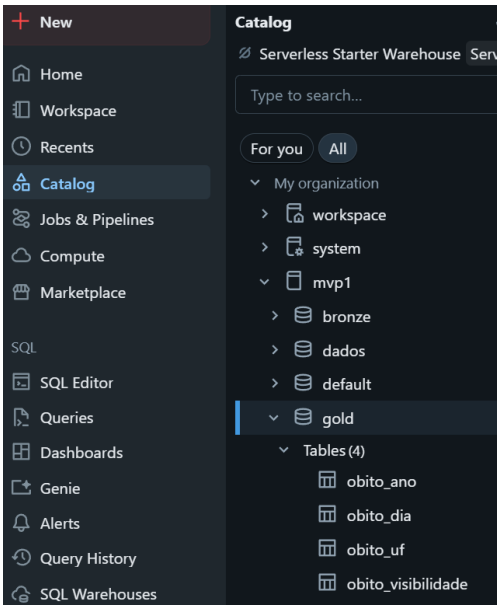


Figura 33: Usando a tabela do schema gold para óbito por visibilidade

```python
import pandas as pd
import matplotlib.pyplot as plt

# Carregando a tabela mvp1.gold.obito_visibilidade como Spark DataFrame
percentage_table = spark.table("mvp1.gold.obito_visibilidade")
percentage_pd = percentage_table.toPandas()
percentage_pd.set_index('visibilidade', inplace=True)

# Removendo a linha com índice 'None' ou valores nulos
percentage_pd = percentage_pd[percentage_pd.index.notnull()]
percentage_pd = percentage_pd[percentage_pd.index != 'None']

# Plotando gráfico de barras compostas (empilhadas)
percentage_pd.plot(kind='bar', stacked=False)
plt.title('Gráfico de Barras Compostas: Óbito por Visibilidade')
plt.xlabel('Visibilidade')
plt.ylabel('Quantidade')
plt.legend(title='Óbito', loc='upper right')
plt.yticks(rotation=0)
plt.show()
```

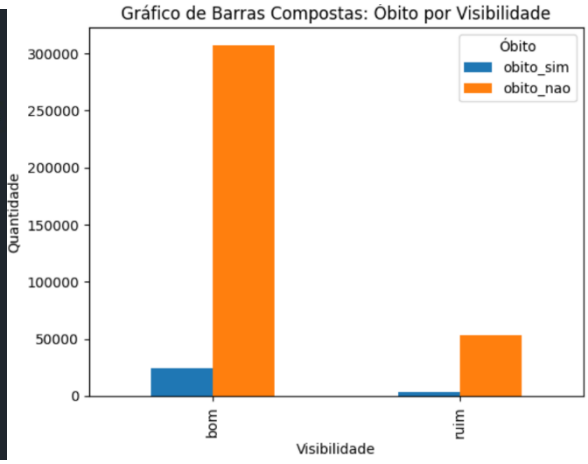


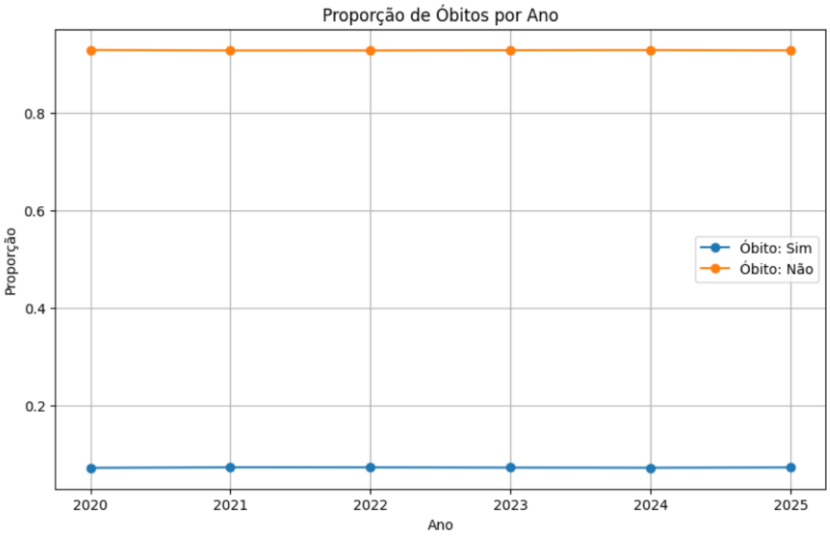
Figura 34: Usando a tabela do schema gold para proporção de óbitos por ano

```python
import pandas as pd
import matplotlib.pyplot as plt

# Carregando a tabela como Spark DataFrame
obito_ano_df = spark.table("mvp1.gold.obito_ano")
obito_ano_pd = obito_ano_df.toPandas()

# Calculando proporções
obito_ano_pd['total'] = obito_ano_pd['obito_sim'] + obito_ano_pd['obito_nao']
obito_ano_pd['proporcao_sim'] = obito_ano_pd['obito_sim'] / obito_ano_pd['total']
obito_ano_pd['proporcao_nao'] = obito_ano_pd['obito_nao'] / obito_ano_pd['total']

# Plotando gráfico de linha para proporções
plt.figure(figsize=(10,6))
plt.plot(obito_ano_pd['ano'], obito_ano_pd['proporcao_sim'], marker='o', label='Óbito: Sim')
plt.plot(obito_ano_pd['ano'], obito_ano_pd['proporcao_nao'], marker='o', label='Óbito: Não')
plt.title('Proporção de Óbitos por Ano')
plt.xlabel('Ano')
plt.ylabel('Proporção')
plt.legend()
plt.grid(True)
plt.show()
```



Como, por uma questão de escala, as linhas eram bastante distantes, aparentavam ser praticamente constantes, a figura 35 exibe apenas a quantidade de óbitos por ano, aonde fica visível o comportamento dos dados. Destaca-se a tendência crescente do número de ocorrências com vítimas fatais entre 2020 e 2025. A queda de 2024 para 2025 pode ocorrer por conta do ano de 2025, que não apresentou os dados de todos os meses, visto que o arquivo foi obtido ainda em outubro.

Figura 35: Usando a tabela do schema gold para quantidade de óbitos por ano

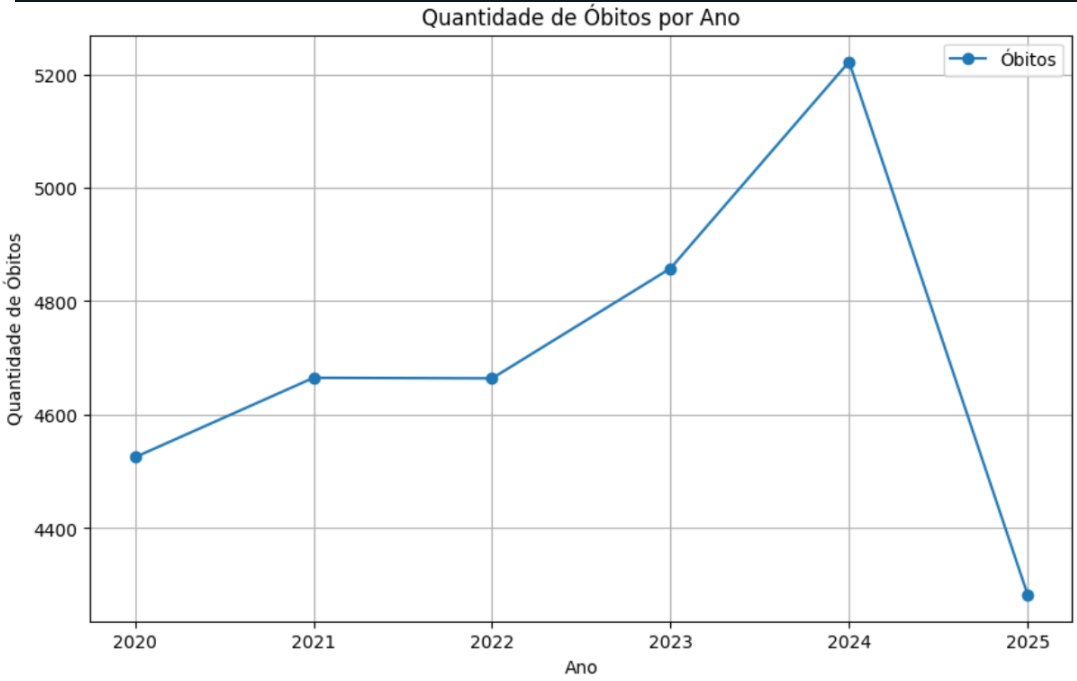
2 minutes ago (29s) 26:

```
import pandas as pd
import matplotlib.pyplot as plt

# Carregando a tabela como Spark DataFrame
obito_ano_df = spark.table("mvp1.gold.obito_ano")
obito_ano_pd = obito_ano_df.toPandas()

# Plotando gráfico de linha apenas para óbitos por ano
plt.figure(figsize=(10,6))
plt.plot(obito_ano_pd['ano'], obito_ano_pd['obito_sim'], marker='o', label='Óbitos')
plt.title('Quantidade de Óbitos por Ano')
plt.xlabel('Ano')
plt.ylabel('Quantidade de Óbitos')
plt.legend()
plt.grid(True)
plt.show()
```

See performance (1)



3. Resultados

Para o resultado final, além das descritivas e do teste qui-quadrado, foi ajustado um modelo de regressão logística binário para a ocorrência de óbito em relação a um conjunto de possíveis fatores de risco. Os testes qui-quadrados foram apresentados nas seções anteriores, especificamente nas seções anteriores (camada silver e gold). O código para ajuste da regressão logística está nas figuras 36 e 37.

Além de ajustar o modelo, a tabela final (intitulada logit_resultados a partir do data frame resultados_reg_log) foi salva no CATALOG mvp1, no SCHEMA gold, sendo considerado o resultado mais importante para o cumprimento do objetivo. A acurácia do modelo foi de 0,93, indicando uma boa capacidade preditiva. Inicialmente o banco de dados foi dividido em treinamento (70%) e teste (30%). A figura 38 exibe todas as tabelas e resultados gerados no estudo em cada camada. A figura 39 mostra as categorias de referência para cada variável explicativa do modelo.

Figura 36: Código em python para o ajuste da regressão logística para a ocorrência de óbito em relação às variáveis explicativas nos sinistros em rodovias federais (2020-2025).

01:01 PM (18m) 27

```python
# 1. Importar bibliotecas
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
import numpy as np
import scipy.stats as stats

# 2. Carregar dados do banco mvp1.silver.datatran_clean
df = spark.table("mvp1.silver.datatran_clean").toPandas()
features = ['visibilidade', 'fase_dia', 'dia_semana', 'causa_acidente', 'uso_solo', 'tipo_pista']
X = pd.get_dummies(df[features], drop_first=True)
y = (df['obito'] == 'sim').astype(int)

# 3. Dividir dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

# 4. Instanciar e treinar o modelo
modelo = LogisticRegression(max_iter=1000)
modelo.fit(X_train, y_train)
print(modelo)

# 5. Fazer previsões
previsoes = modelo.predict(X_test)
probabilidades = modelo.predict_proba(X_test)

# 6. Avaliar o modelo (exemplo com acurácia)
acuracia = accuracy_score(y_test, previsoes)
print(f"Acurácia do modelo: {acuracia:.2f}")
```

Figura 37: Código em python para estimação da regressão logística para a ocorrência de óbito em relação às variáveis explicativas em acidentes em rodovias federais de 2020 a 2025 (PARTE 2).

```python
# 7. Resultados das estimativas, erro padrão, teste e odds ratios
params = modelo.coef_[0]
intercept = modelo.intercept_[0]
odds_ratios = np.exp(params)
# Erro padrão
X_design = np.hstack([np.ones((X_train.shape[0], 1)), X_train.values])
preds = modelo.predict_proba(X_train)[:,1]
V = np.diag(preds * (1 - preds))
cov_logit = np.linalg.inv(X_design.T @ V @ X_design)
std_err = np.sqrt(np.diag(cov_logit))[1:]  # Ignora intercepto
# Teste z e p-valor
z_scores = params / std_err
p_values = 2 * (1 - stats.norm.cdf(np.abs(z_scores)))
# Exibir resultados
resultados = pd.DataFrame({
    'variavel': X.columns,
    'estimativa': params,
    'erro_padrao': std_err,
    'z': z_scores,
    'p_valor': p_values,
    'odds_ratio': odds_ratios})
display(resultados)
# Salvar resultados na camada gold do catalog mvp1
resultados_reg_log = spark.createDataFrame(resultados)
resultados_reg_log.write.mode('overwrite').saveAsTable("mvp1.gold.logit_resultados")
# 8. Prever para novos dados
nova = pd.DataFrame({'visibilidade': ['bom'], 'fase_dia': ['Dia'], 'dia_semana': ['Domingo'],
                     'causa_acidente': ['Falta de atenção'], 'uso_solo': ['Urbano'], 'tipo_pista': ['Simples']})
nova_dummies = pd.get_dummies(nova)
nova_dummies = nova_dummies.reindex(columns=X.columns, fill_value=0)
previsao_nova = modelo.predict(nova_dummies)
probabilidade_nova = modelo.predict_proba(nova_dummies)
print(f"Previsão para novo exemplo: {previsao_nova[0]}, Probabilidades: {probabilidade_nova[0]}")
```

Observando a tabela final, quando a visibilidade é ruim, as chances de um óbito são aproximadamente 11,66% menores do que quando a visibilidade é boa. Já a fase do dia é significativo e reduz em 36,16% a chance de ocorrência de acidente fatal para o anoitecer em relação ao amanhecer, enquanto a plena noite diminui apenas em 7,3%. Todos os dias da semana foram comparados com domingo (base de referência), de modo que todos foram fatores de proteção, ou seja, diminuem a chance de ocorrência de óbito. Isto é, o domingo, como visto, é o dia que mais ocorrem óbitos quando comparado aos demais dias.

Outro resultado importante está associado ao tipo de pista. A pista simples apresenta mais de 76% de chance a mais de ocorrer óbito em relação à pista dupla, enquanto a pista múltipla 0,76% de chance a menos ter risco de óbito em relação a pista dupla. A ultrapassagem indevida aumenta em mais do dobro de chance de ocorrência de

óbito, enquanto transitar na contramão aumenta em mais de 4 vezes a chance de óbito. Iluminação deficiente também aumenta em dobro a chance de óbito.

Figura 38: Confirmação das tabelas criadas em cada camada da arquitetura medalhão

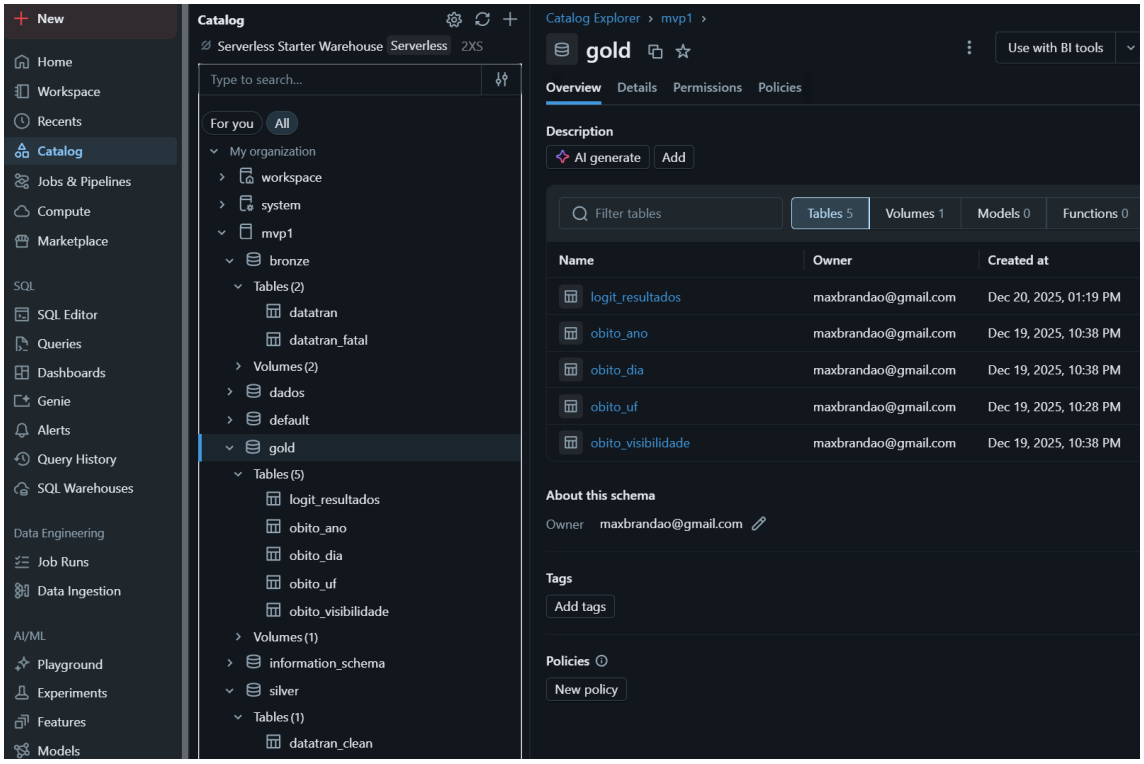
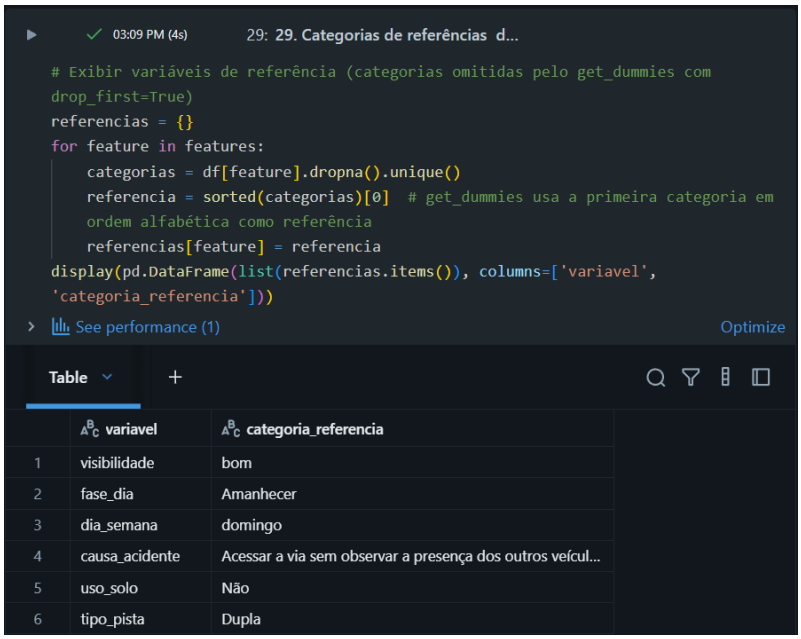


Figura 39: Categorias de referência adotadas para cada variável explicativa

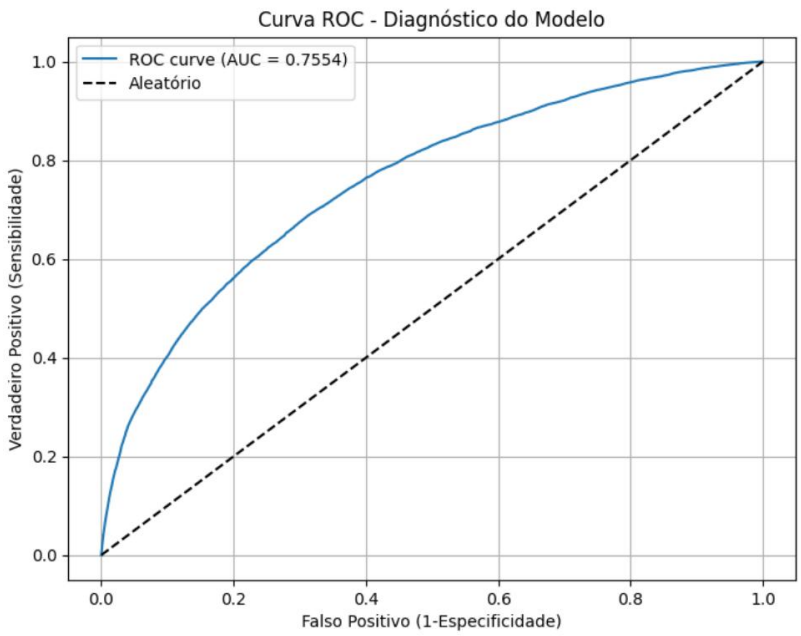


Além da acurácia do modelo, que foi de 93%, podemos obter a curva característica de operação (ROC), que reflete a comparação entre a taxa de verdadeiro positivo e de falso positivo. A área sob a curva é um dos principais indicadores da regressão logística, de modo que, quanto mais próximo de 1, melhor. A figura 40 traz o código para obtenção da curva e da área sob a curva (AUC), enquanto a figura 41 apresenta a própria curva. Pelo valor obtido, observamos que o modelo é satisfatório e discrimina bem as variáveis em termos de classificação da chance de um acidente ser fatal a partir das variáveis adotadas no modelo.

Figura 40: Comandos para obtenção da curva característica de operação (ROC) do modelo ajustado para predição de acidentes fatais nas rodovias federais de 2020 a 2025

```
# 6. Cálculo da área sob a curva ROC e gráfico da curva ROC
auc = roc_auc_score(y_train, preds)
fpr, tpr, _ = roc_curve(y_train, preds)
plt.figure(figsize=(8,6))
plt.plot(fpr, tpr, label=f'ROC curve (AUC = {auc:.4f})')
plt.plot([0,1], [0,1], 'k--', label='Aleatório')
plt.xlabel('Falso Positivo (1-Especificidade)')
plt.ylabel('Verdadeiro Positivo (Sensibilidade)')
plt.title('Curva ROC - Diagnóstico do Modelo')
plt.legend()
plt.grid(True)
plt.show()
print(f"Área sob a curva ROC (AUC): {auc:.4f}")
```

Figura 41: Curva característica de operação (ROC) e área sob a curva ROC do modelo ajustado para predição de acidentes fatais nas rodovias federais de 2020 a 2025



4. Conclusão

O objetivo desse estudo foi avaliar fatores de risco (ou de proteção) associados à ocorrência de acidentes com óbitos nas rodovias federais de todo o Brasil de 2020 a 2025 registrados pela PRF. O modelo de regressão logística mostrou-se apropriado para identificar os fatores associados as chances de letalidade nos acidentes de trânsito nas rodovias federais.

De acordo as técnicas empregadas para o diagnóstico do modelo, mostrou-se apropriado, verificado através dos Testes: Wald, e o número de AUC da Curva Característica de Operação (ROC), apresentou uma acurácia satisfatória de 93% para a classificação. Os resultados do estudo apontam contribuição significativa dos fatores de risco para o desfecho óbito. As razões de chances obtidas no modelo contribuíram para entender o aumento ou redução das chances de óbito em uma ocorrência de acordo com os níveis de cada variável.

Dirigir em condições de visibilidade ruim diminui as chances de ocorrências fatais, pôde-se ver também a diminuição dos riscos de óbito quando conduz o veículo em pleno dia. Apontou-se que as razões de chances de ocorrências fatais para quem dirige sob efeito de álcool, são maiores se comparada aos condutores que dirigem sóbrios, no entanto mesmo cientes de todos os riscos e consequências muitos continuam arriscando suas próprias vidas e a dos outros.

Os resultados da análise revelam que se deve ter um cuidado especial a velocidade incompatível, em nosso estudo, dirigir acima da velocidade permitida poder elevar em até 5 vezes as chances de morte relacionados com esta causa, segundo o Anuário Estatístico da Polícia Rodoviária Federal, em 2021. Quanto maior a velocidade na hora do sinistro, maior a letalidade. Os resultados mostram que a chance de fatalidade em acidentes cuja causa é ultrapassagem indevida é extremamente elevada se comparada as demais, quando se ultrapassa em pista simples, momento em que o condutor aumenta a velocidade de seu veículo ocorre a invasão da faixa de sentido contrário, daí pode acontecer a colisão frontal em local e momento inapropriados causando mortes ou sequelas permanentes.

Em resumo, podemos concluir que os fatores investigados influenciam de forma significativa a ocorrência de óbitos nas rodovias federais. Dentre elas, visibilidade foi a que contrariou o esperado, pois a visibilidade ruim age como fator de proteção, talvez

porque o condutor, ao perder a visibilidade, redobre a atenção, reduza a velocidade e aumente a cautela na condução.

5. Dificuldades e trabalhos futuros

Apesar de ser bacharel em estatística e atuar como tal, utilizava prioritariamente a linguagem R para manuseio dos dados, bem como para transmitir para os discentes da universidade Federal do Piauí, onde sou docente do curso de Estatística. Porém, estava restrito a obter os dados no formato de origem, importar para o R e fazer as análises. Havia trabalhado com sql há mais de 10 anos, bem como usava python para poucas atribuições. Então, a priori tive dificuldade para trabalhar todo o contexto de banco de dados e da arquitetura em nuvem, assim como para trabalhar com a migração entre as linguagens e todas as definições associados ao objetivo do estudo.

Ter realizado a atividade da MVP foi uma atividade foi uma experiência muito enriquecedora para minha atuação, tanto como professor, como para estatístico. Me senti apto a realizar atividades específicas em outras linguagens, que antes executava apenas em R. Como trabalho futuro, seria interessante usar outras abordagens mais recentes, como redes neurais, que já estimei modelos de redes neurais artificias perceptron e em casos de redes recorrentes, mas apenas no R, e não em python. Como sugestão para trabalhos futuros, também recomenda-se incluir mais variáveis no modelo, como idade do envolvido, tipo de acidente, tipo de veículo, traçado da via para investigar se trechos em reta tem influência em acidentes fatais ou ainda pode-se trabalhar por estados da Federação.

Referências

OMS (Organização Mundial da Saúde) Global status report on road safety 2018. Disponível em: https://www.who.int/publications/i/item/9789241565684 Acesso em: 06 mar. de 2023

OMS. (Organização Mundial da Saúde) Década de Ação pela Segurança no Trânsito 2021-2030. Disponível em: https://www.paho.org/pt/noticias/28-10-2021-oms-lanca-decada-acao-pela-seguranca-no-transito-2021-2030. Acesso em: 03 fev. de 2023.

PRF (POLÍCIA RODOVIÁRIA FEDERAL). Anuário 2021. Distrito federal, 2021. disponível em: https://www.gov.br/prf/pt-br/acesso-a-informacao/dados-abertos/diest-arquivos/copy_of_anuario-2021_final.html Acesso em: 06 mar. 2023

PRF (POLÍCIA RODOVIÁRIA FEDERAL). Acidentes. Disponível em: https://www.gov.br/prf/pt-br/acesso-a-informacao/dados-abertos/dados-abertos-acidentes. Acesso em: 2022

AGRESTI, A. An introduction to categorical data analysis. 3nd ed. Hoboken, New Jersey: John Wiley Sons, 2019.

R Core Team. R: A language and environment for statistical computing. R Foundation for Statistical Computing, Vienna, Austria, 2022. Disponível em: https://www.R-project.org/. Acesso em: 05 set. 2022.

ROSEMBERG, V. T. Estudo sobre a Incidência de Acidentes Fatais no Trânsito do Distrito Federal. Orientador: Prof. Leandro Tavares Correia. 2018.58 f. Trabalho de Conclusão de Curso– Universidade de Brasília– UnB Departamento de Estatística, Brasília- DF, 2018. Disponível em: https://bdm.unb.br/bitstream/10483/24356/1/2018_VitorTeofiloRosemberg_tcc.pdf. Acesso em: 06 mar. 2023.

TRANSPORTE, C. N. do. Painel CNT de Acidentes Rodoviários-Principais dados-2021. CNT, 2021. Disponível em: https://www.cnt.org.br/agencia-cnt/acidentes-e-mortes-nas-rodovias-federais-aumentam-em-2021. Acesso em: 06 mar. 2023.

MORAIS, Marcleiton Ribeiro et al. Letalidade do acidente de trânsito na modernista Palmas/TO: uma abordagem econométrica. Informe Gepec, v. 18, n. 1, p. 156-176, 2014.

MINGOTI, S. A Análise de Dados Através Métodos de Estatística Multivariada: Uma Abordagem Aplicada. 4. ed. Belo Horizonte: Editora UFMG, 2020.

ONSV (Observatório Nacional de Segurança Viária). Acidentes de trânsito são uma questão de saúde pública no Brasil Disponível em: https://www.onsv.org.br/comunicacao/artigos/ acidentes-de-transito-sao-uma-questao-de-saude-publica-no-brasil. Acesso em: 23 mar. 2023.